End2Race: Efficient End-to-End Imitation Learning for Real-Time F1Tenth Racing

Zhijie Qiao[1,†], Haowei Li[1,†], Zhong Cao[1], Henry X. Liu[1,2,*]

***Abstract*— F1Tenth is a widely adopted reduced-scale platform for developing and testing autonomous racing algorithms, hosting annual competitions worldwide. With high operating speeds, dynamic environments, and head-to-head interactions, autonomous racing requires algorithms that diverge from those in classical autonomous driving. Training such algorithms is particularly challenging: the need for rapid decision-making at high speeds severely limits model capacity. To address this, we propose End2Race, a novel end-to-end imitation learning algorithm designed for head-to-head autonomous racing. End2Race leverages a Gated Recurrent Unit (GRU) architecture to capture continuous temporal dependencies, enabling both short-term responsiveness and long-term strategic planning. We also adopt a sigmoid-based proximity encoding that transforms raw LiDAR scans into a distance-sensitive representation, facilitating effective model training and convergence. The algorithm is extremely efficient, achieving an inference time of less than 0.5 milliseconds on a consumer-class GPU. Experiments in the F1Tenth simulator demonstrate that End2Race achieves a 94.2% safety rate across 2,400 overtaking scenarios, each with an 8-second time limit, and successfully completes overtakes in 59.2% of cases. This surpasses previous methods and establishes ours as a leading solution for the F1Tenth racing testbed. Code is available at https://github.com/michigan-traffic-lab/End2Race.**

I. Introduction

Autonomous racing serves as a compelling proving ground for advancing autonomous vehicle systems. It combines perception, planning, and control under high-speed, interactive, and computationally constrained conditions. Enhancing algorithmic capabilities in this domain is directly relevant to the broader development of autonomous driving. F1Tenth [1], a 1/10-scale vehicular platform, provides an accessible testbed for autonomous racing education and research. Equipped with a 2D LiDAR [2] or RGB-D camera [3] for perception, the vehicle can reach speeds of up to 17.9 m/s [4]. Each year, competitions are held at venues worldwide and attract broad participation from the community.

Existing approaches for autonomous racing can be broadly grouped into map-based, reactive, and learning-based methods. Among them, map-based pipelines are the most widely used, relying on a pre-constructed map to follow a globally optimized raceline. During preprocessing, the track layout is mapped using SLAM [5]. A minimum-curvature raceline is then computed offline to serve as the global reference path [6]. During racing, particle-filter localization [7] tracks the vehicle's position along the track, enabling a lattice planner [8] to evaluate feasible local trajectories. A tracking controller, such as Pure Pursuit [9] or model predictive control (MPC) [10], then follows the selected path. However, map-based methods have two limitations. First, they require preprocessing and cannot operate on previously unseen tracks. Second, the multi-stage localization and planning pipeline incurs a delay of approximately 30 ms on onboard hardware [11], [12]. With additional sensor latency, the vehicle can travel up to a full vehicle length before updated control commands take effect. Although non-critical under most conditions, this delay becomes consequential when a high-speed straight leads directly into a sharp curve, leaving the vehicle susceptible to boundary collisions. Such collisions are widely observed in F1Tenth competitions.

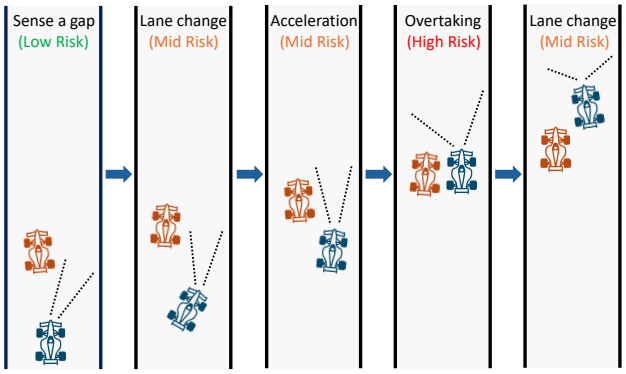


Fig. 1. Simple overtake scenario illustration.

Reactive methods provide a simpler, rule-based alternative by acting directly on current sensor measurements. Their map-free design avoids localization and planning, reducing computational cost while allowing operation on unknown tracks. The Follow-the-Gap algorithm [13], for example, identifies the largest free region and steers toward its midpoint. However, these methods operate instantaneously, as each decision is based on an isolated observation with limited temporal context, leading to planning inconsistencies and control instability [14].

Learning-based methods have also been widely proposed for F1Tenth. Early work focused primarily on single-vehicle navigation [15], [16], whereas more recent studies have shifted toward head-to-head competition. However, existing approaches either use only static obstacles during training

This research was partially funded by the DARPA TIAMAT Challenge (HR0011-24-9-0429).

[1]Z. Qiao, H. Li, Z. Cao, and H. X. Liu are with the Department of Civil and Environmental Engineering, University of Michigan, Ann Arbor, MI 48109, USA.

[2]H. X. Liu is also with University of Michigan Transportation Research Institute, Ann Arbor, MI 48109, USA.

†These authors contributed equally to this work.

*Corresponding author: Henry X. Liu (henryliu@umich.edu).

before being tested directly against dynamic opponents [17], rely on a small set of human demonstrations and evaluation scenarios [18], or operate at low racing speeds [19].

In high-speed head-to-head competition, overtaking is a key determinant of the race outcome and presents an intrinsically challenging problem. Figure 1 illustrates a simplified five-stage overtaking process. First, the ego vehicle identifies a suitable opening, executes a sharp lane change to align with it, and accelerates rapidly to catch up with the opponent. During the pass, the vehicles race side by side with minimal safety margins. After completing the pass, the ego vehicle returns to the track center to regain lateral clearance. Successful overtaking therefore depends on continuous decision-making, precise maneuver execution, rapid adaptation to the opponent's motion, and minimal sensing-to-control latency. This requires both sufficient model capacity and low inference latency, two conflicting demands that existing methods fail to reconcile.

To address these limitations, we propose **End2Race**, an end-to-end learning framework for real-time, head-to-head autonomous racing. The framework generates diverse overtaking scenarios across dynamic racing conditions, enabling systematic policy training and evaluation at scale. Utilizing this framework, we train a racing policy based on a Gated Recurrent Unit (GRU) architecture [20]. The policy takes a 360-degree 2D LiDAR scan and the current ego speed as inputs and directly outputs steering and speed commands, achieving an inference time below 1 ms on an F1Tenth onboard computer. Training follows a two-stage pipeline that initializes the policy through imitation learning and then fine-tunes it using reinforcement learning [21]. Across both training and test tracks, the resulting policy demonstrates high racing speeds, strong safety performance, and competitive overtaking capabilities.

The main contributions of this work are as follows:

1) We introduce End2Race, an end-to-end learning framework for autonomous racing that supports scalable overtaking scenario generation, policy training, and benchmark evaluation.
2) We design a computationally efficient recurrent policy network with a sensing-to-control latency below 1 ms, supporting real-time operation at high racing speeds.
3) We show that our two-stage training pipeline combining imitation learning with reinforcement learning fine-tuning yields strong generalization across novel tracks, delivering safe and competitive racing behavior.

II. Related Work

A. Imitation Learning

Imitation learning (IL) trains a driving policy from expert demonstrations. Sun et al. [15] generated demonstrations using a Pure Pursuit controller to evaluate BC [22], DAgger [23], HG-DAgger [24], and EIL [25], finding that all four policies completed the training track but failed to generalize to an unseen track. Paluch et al. [26] trained a feedforward policy to imitate nonlinear model predictive control along an offline raceline, achieving faster lap times than Pure Pursuit. TinyLidarNet [17] trained a compact convolutional network on static obstacle demonstrations and then evaluated it directly against dynamic opponents. Zhang and Loidl [18] trained dense and recurrent policies from human overtaking demonstrations, but relied on limited training data and evaluated across only a few scenarios.

B. Model-Free Reinforcement Learning

Model-free RL optimizes control policies through environmental interaction to maximize cumulative reward. Hamilton et al. [27] compared DDPG [28], SAC [29], and BC [22], finding that RL policies achieved faster lap times but incurred higher collision rates. Steiner et al. [19] trained a TD3 [30] policy for head-to-head racing, demonstrating that opponent-inclusive training improves overtaking success against diverse baselines. However, their deep feedforward policy network introduced substantial inference delays, compromising real-time responsiveness and forcing operating speeds below 2 m/s. Such velocities fall far below practical racing limits, leaving the challenges of high-speed multi-vehicle competition largely unaddressed.

C. Model-Based Reinforcement Learning

Model-based RL learns a predictive model of the environment to optimize control policies through imagined rollouts. For instance, Dreamer [31] learns a recurrent state-space model and trains actor-critic networks entirely within a latent space. Brunnbauer et al. [16] applied Dreamer to autonomous racing, outperforming model-free baselines in sample efficiency and lap completion.

D. Hybrid Learning Methods

Hybrid methods integrate classical planning or control modules into learned autonomy pipelines. Dwivedi et al. [32] augmented an RL tracking controller with reference trajectories from a global planner, combining low-level continuous control with high-level path guidance. Bhargav et al. [33] learned overtaking success probabilities across track segments to dynamically select discrete lateral planning modes. Similarly, Trumpp et al. [34] coupled residual reinforcement learning with an artificial potential field planner to support mapless multi-agent racing.

III. Methodology

We develop End2Race using the F1TENTH simulator [1], which provides high-fidelity vehicle dynamics and accurate 2D LiDAR simulation. Four tracks are selected reflecting real-world circuit layouts: Austin, Hockenheim, MoscowRaceway, and Nuerburgring. Among these, Austin is used for training, while the remaining three are reserved for evaluation. For each track, an automated workflow generates diverse overtaking scenarios, each pairing an ego vehicle with a leading opponent. Using demonstrations provided by a map-based expert, we first train a GRU network through Behavioral Cloning (BC) [22], and then fine-tune the resulting policy through Proximal Policy Optimization (PPO) [35]. The following sections describe each stage in detail (Fig. 2).

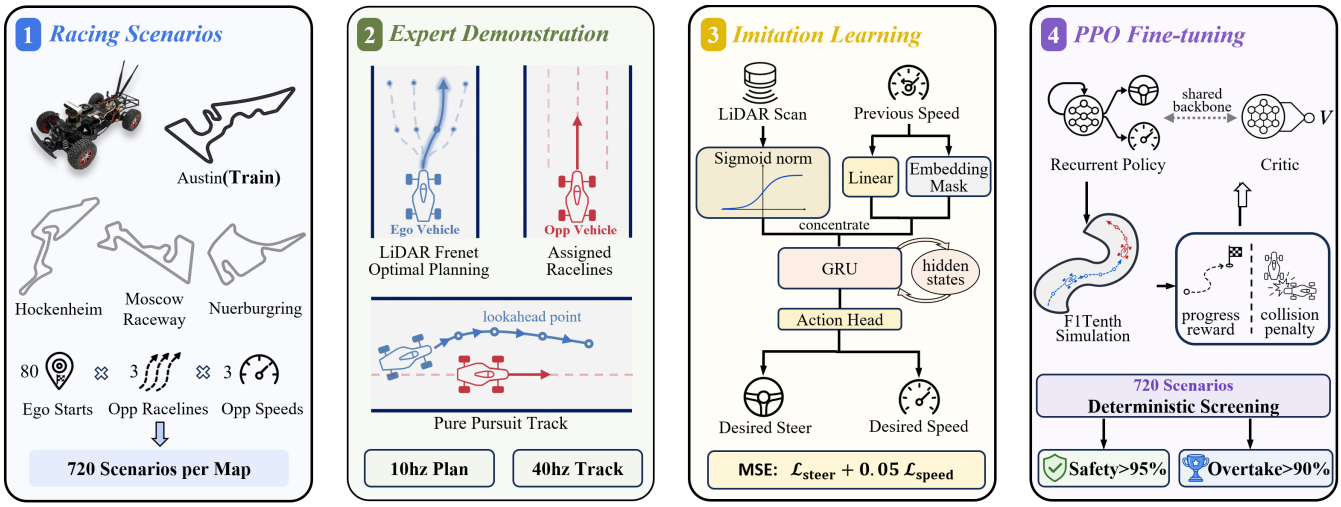


Fig. 2. End2Race system overview: (a) racing scenarios, (b) expert demonstration, (c) imitation learning, and (d) reinforcement learning.

A. Overtaking Scenario Setup

Each overtaking scenario pairs an ego vehicle with an opponent initially positioned ahead of it and has a duration of 8 seconds. Vehicle placement is expressed in Frenet coordinates [36], in which the longitudinal direction measures progress around the circuit and the lateral direction specifies position across the track width. We specify each scenario using four parameters, $(s_0,\, \Delta s_0,\, r_{\text{opp}},\, \alpha_{\text{opp}})$.

1) Ego Starting Position (s_0): For each track, we define N_s ego starting positions equally spaced in the longitudinal direction, with $N_s = 80$ fixed across all tracks. For example, on the 420 m Austin track, adjacent starting positions are separated by approximately 5.2 m.

2) Longitudinal Gap (Δs_0): The longitudinal gap Δs_0 specifies how far the opponent is positioned ahead of the ego vehicle along the track at the start of a scenario. A small gap provides insufficient starting space for the ego vehicle to maneuver, while a large gap requires the ego vehicle to spend more time catching the opponent, leaving less time for interaction. We therefore set $\Delta s_0 = 3$ m to balance these considerations.

3) Opponent Raceline (r_{opp}): Beyond longitudinal placement, the lateral positions of the ego vehicle and opponent are critical to the overtaking geometry. We represent this lateral configuration using the opponent raceline r_{opp}, which assigns the opponent to one of three reference paths. Raceline generation follows the framework of Heilmeier et al. [6], positioning left, center, and right paths at 35%, 50%, and 65% across the track width from the left boundary. On the Austin circuit, this yields approximately 0.25 m of lateral separation between the center and each side raceline. An opponent on the center raceline restricts lateral clearance on both sides and is harder to pass, whereas one on a side raceline leaves a wider passing corridor. Meanwhile, the ego vehicle always uses the center raceline as its reference, but may temporarily deviate from it as needed.

4) Opponent Speed Scale (α_{opp}): Each reference raceline provides an optimized speed profile based on track curvature and vehicle dynamics. Here, we set a maximum speed of 7.5 m/s, as empirical testing showed that higher values risk compromising tracking and control stability. Crucially, if the opponent follows this profile directly, the ego vehicle sharing the same profile cannot catch up or overtake. To create overtaking opportunities, we scale down the opponent's speed by a factor of $\alpha_{\text{opp}} \in \{0.4, 0.6, 0.8\}$. Scales below 0.4 make the opponent unrealistically slow, while those above 0.8 often prevent the ego vehicle from catching up in time. Meanwhile, the ego vehicle always operates under the full speed limit.

Combining 80 ego starting positions s_0, a fixed longitudinal gap Δs_0, three opponent racelines r_{opp}, and three opponent speed scales α_{opp} yields 720 scenarios per track. During each 8-second scenario, the opponent passively tracks its assigned raceline with a Pure Pursuit controller. It does not react to the ego vehicle, nor does it actively engage in adversarial behavior, such as defensive blocking or erratic speed changes [37].

B. Expert Demonstration

We design a map-based lattice planner to generate overtaking demonstrations for the ego vehicle. The lattice planner, adapted from the Frenet Optimal Trajectory framework [38], converts each LiDAR scan into obstacle points and generates candidate trajectories across a range of lateral positions and terminal speeds. It then discards candidates that violate dynamic constraints and selects the trajectory τ that minimizes the cost $\mathcal{J}(\tau)$:

$$\mathcal{J}(\tau) = \frac{1}{N}\sum_{i=1}^{N}\left(1 - \frac{v_i}{v_{\max}}\right) + \lambda_c \max_i \left[\max\left(0, 1 - \frac{d_i}{d_c}\right)\right]^2 \tag{1}$$

where N is the number of future trajectory points sampled at 0.1 s intervals ($N = 6$); v_i is the projected vehicle speed at point i, favoring trajectories with higher mean speeds; $v_{\max} = 7.5$ m/s is the speed limit; d_i is the distance

Fig. 3. Expert-planner failure during overtaking, shown in temporal order from (I) to (III). Blue denotes the ego vehicle and its planned trajectory, and red denotes the opponent. In this scenario, the opponent follows the right raceline with a speed scale of 0.8. (I) The ego approaches the opponent, preparing to overtake. (II) The vehicles travel side by side. (III) The ego returns to the center raceline prematurely, leading to a sideswipe collision.

from point i to the nearest obstacle; $d_c = 0.5$ m is the clearance threshold for applying the collision penalty; and $\lambda_c = 5$ balances vehicle speed and obstacle clearance. The ego vehicle then tracks the selected trajectory using a Pure Pursuit controller.

C. Training Dataset

We collect 720 eight-second overtaking demonstrations on the Austin track. Training data are recorded at 40 Hz to reflect the onboard LiDAR-to-control update cycle, yielding 320 observation-action pairs per scenario. Among these scenarios, 620 (86.1%) result in successful overtakes, 23 (3.2%) in collision-free car-following, and 77 (10.7%) in collisions. These collisions stem primarily from the lattice planner's forward-only obstacle evaluation, which does not account for a moving opponent alongside or behind the ego vehicle. Consequently, the planner may return to the center raceline before the ego vehicle has fully cleared the opponent. Collision risk increases as the speed difference between the two vehicles decreases, with most failures occurring at an opponent speed scale of 0.8. Figure 3 illustrates one such failure: during overtaking, the planned ego trajectory returns toward the center raceline while the vehicles still overlap longitudinally, resulting in a sideswipe collision.

These collisions can be mitigated by incorporating explicit opponent trajectory prediction and interaction modeling, as demonstrated in Predictive Spliner [39]. However, such approaches remain fundamentally constrained by hand-crafted heuristics and rigid safety margins that struggle to generalize across diverse head-to-head overtaking scenarios. We instead exclude the 77 collision scenarios from BC training and rely on downstream PPO fine-tuning (Sec. III-E) to learn safe interactive behavior.

D. Model Architecture

We adopt an end-to-end policy that maps sensory inputs and the current vehicle state directly to control outputs. At each timestep t, the policy receives a 360-degree 2D LiDAR scan comprising 180 beams at an angular resolution of 2 degrees. The current ego speed is also provided as input to maintain dynamic consistency across consecutive model predictions. The policy outputs the desired vehicle speed in meters per second and the steering angle in radians. To capture temporal dependencies in ego and opponent motion, we adopt a GRU architecture that employs reset and update gates to selectively retain relevant history and incorporate new observations over time. In End2Race, the hidden state summarizes previous observations and provides temporal context for each control prediction.

1) LiDAR Proximity Encoding: Research by Bufacchi and Iannetti shows that human interaction with surrounding space is proximity-dependent and that behavioral responses strengthen progressively as object distance decreases [40]. The same principle can be applied to autonomous racing, as the influence of track boundaries and competing vehicles on ego motion similarly depends on their distance. To emulate this aspect of human spatial cognition in our model design, we transform each LiDAR beam using a sigmoid function that maps each raw scan value, $x \in [0, \infty)$, to a proximity-encoded value within $[0, 1]$:

$$\sigma(x) = \left(-\frac{1}{1 + e^{-kx}} + 1 \right) \cdot 2 \tag{2}$$

Here, x denotes the distance measured by a single LiDAR beam, $\sigma(x)$ denotes its normalized value, and k controls the effective sensing range. We set $k = 0.3$, for which the normalized value decreases to 0.1 at approximately 10 m (Fig. 2(c)), matching the maximum range of the onboard LiDAR sensor. With this design, the encoded value increases rapidly and nonlinearly as obstacles become closer, while asymptotically approaching zero as they become more distant. The resulting representation enables the model to learn effective driving behavior, as demonstrated by the experimental results in Sec. IV. The ablation study further shows that alternative LiDAR normalization methods can converge numerically without yielding a viable driving policy.

2) Ego-Speed Embedding: To maintain dynamic consistency across consecutive control predictions, we project the current ego speed v_t through a linear layer into a 30-dimensional embedding $\psi(v_t)$. To prevent shortcut learning [41], in which the model simply copies the current ego-speed input into its desired-speed prediction, we randomly mask the projected ego speed using a learnable embedding:

$$\widetilde{\psi}(v_t) = \begin{cases} \psi(v_t), & \text{with probability } 1 - p, \\ \mathbf{e}_{\text{mask}}, & \text{with probability } p, \end{cases} \tag{3}$$

Here, we set $p = 0.2$, encouraging the model to determine an appropriate desired speed from the LiDAR observation when the ego-speed embedding is masked and to incorporate the current ego speed when it is available.

3) Recurrent Control Prediction: We concatenate the 180-dimensional proximity-encoded LiDAR representation with the 30-dimensional ego-speed embedding to form a 210-dimensional input vector. At each timestep, the GRU combines this vector with the preceding hidden state $\mathbf{h}_t$ to produce the updated hidden state $\mathbf{h}_{t+1}$:

$$\mathbf{h}_{t+1} = \text{GRU}_\theta \left([\sigma(x_{t,i})]_{i=1}^{180} \| \widetilde{\psi}(v_t), \mathbf{h}_t \right), \tag{4}$$

where the hidden-state dimension is 420, twice that of the input. An action head ($420 \to 128 \to 2$) then maps $\mathbf{h}_{t+1}$

to the next steering-angle and desired-speed commands. In total, the policy contains 850,556 trainable parameters.

E. Policy Training

Policy training proceeds in two stages. We first use BC to learn car-following and overtaking behavior from expert demonstrations, producing a functional end-to-end racing policy whose performance remains bounded by that of the expert. RL then addresses this limitation by refining the policy's racing behavior through direct environment interaction.

1) Behavioral Cloning: Each 8-second scenario is processed as a complete input sequence, generating a control prediction at every timestep and aggregating the losses across the full temporal horizon. The loss function $\mathcal{L}$ combines the speed and steering mean squared error (MSE) terms, with a weight of 0.05 applied to the speed term to balance their different scales:

$$\mathcal{L} = 0.05\,\mathcal{L}_{\text{speed}} + \mathcal{L}_{\text{steer}}. \tag{5}$$

2) Reinforcement Learning Fine-Tuning: The trained policy is fine-tuned using PPO [35] through direct interaction with the environment on the Austin track. During rollout, the policy is held fixed while one trajectory is collected for each of the 720 overtaking scenarios. These trajectories form a single batch for policy optimization. Empirical testing shows that the complete batch improves training stability compared with smaller batches and yields steady performance gains throughout training. The associated cost is less frequent policy updates and a longer overall training period.

The reward function r_t takes a straightforward form that combines a raceline-progress reward with a collision penalty. At each control step t, it is computed as:

$$r_t = 0.01 d_t - 3 c_t, \tag{6}$$

Here, d_t is the distance in meters traveled by the ego vehicle along the raceline since the preceding control step. Over a fixed duration, this progress term encourages higher driving speeds. c_t is a binary collision indicator equal to 1 if the ego vehicle collides at step t and 0 otherwise. The weight of 3 makes collision events substantially more consequential than incremental progress. Together, these terms encourage the ego vehicle to prioritize safety while exploiting overtaking opportunities as they arise.

IV. Experiments

This section provides implementation details and reports experimental results on the training track as well as three unseen test tracks. We first present model performance in the single-vehicle setting, showing detailed driving metrics. Next, we evaluate policy behavior across overtaking scenarios spanning all four tracks, analyzing outcomes under diverse conditions. Four representative examples are shown in Fig. 5: two successful overtakes, safe car-following, and a collision.

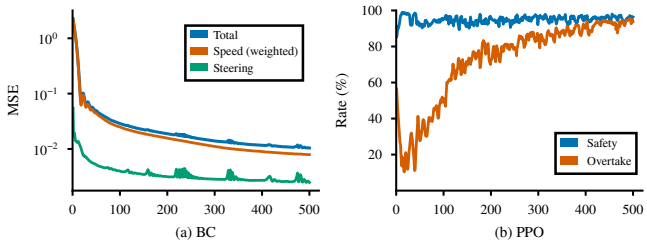


Fig. 4. Training curves for the two-stage policy optimization, with each stage trained for 500 epochs. (a) BC loss, with the speed loss weighted by 0.05. (b) PPO safety and overtake rates on the 720 Austin scenarios.

A. Training Setup

For BC, we train the policy for 500 epochs using the Adam optimizer at a learning rate of 10^{-4}, processing the full dataset as a single batch in each epoch. Training incurs minimal computational cost, requiring less than five minutes on a single NVIDIA L40S GPU. As shown in Fig. 4(a), the training loss decreases rapidly during the early epochs and then gradually converges. Although additional training further reduces loss, empirical testing indicates that doing so leads to overfitting without improving closed-loop driving performance.

For PPO, we fine-tune the policy for 500 epochs and perform a deterministic evaluation after each epoch. The policy network is initialized from the BC checkpoint and paired with a randomly initialized value head. We use a learning rate of 10^{-5} and clip the policy ratio to $[0.8, 1.2]$. PPO training is CPU-intensive because running the simulator online models LiDAR sensing, vehicle dynamics, and multi-vehicle interactions at every timestep. We therefore use a data-center server with 64 CPU cores and four NVIDIA L40S GPUs. Under this configuration, each epoch takes approximately 120 seconds, resulting in a total training time of roughly 16 hours for 500 epochs. Training remains feasible with fewer computational resources at the cost of increased wall-clock time. As shown in Fig. 4(b), the safety rate rises rapidly to nearly 100% during the early epochs, while the overtake rate drops below 20%. This behavior is expected because the large collision penalty initially encourages the policy to adopt slow, conservative car-following behavior. As training progresses, the overtake rate gradually increases while the safety rate remains high, indicating that the policy learns to execute overtaking maneuvers while maintaining strong safety performance.

B. Single-Vehicle Racing

We first evaluate End2Race in single-vehicle racing, which tests the basic requirement of completing a lap safely at high speed. The evaluation consists of one complete lap on each track. Table I (top) compares the mean speed and lap time of three methods: the map-based expert, BC policy, and PPO policy, all of which complete the evaluation without a collision. The map-based expert establishes the rule-based baseline through explicit track knowledge and trajectory planning, achieving collision-free completion while maintaining consistent racing speeds. The map-free BC policy

TABLE I

BENCHMARK EVALUATION: SINGLE-VEHICLE AND HEAD-TO-HEAD RACING PERFORMANCE

Setting	Metric	Austin (Train)			Hockenheim			MoscowRaceway			Nuerburgring		
		Expert	BC	PPO	Expert	BC	PPO	Expert	BC	PPO	Expert	BC	PPO
Single-Vehicle	Mean Speed (m/s)	6.7	6.7	**8.7**	6.8	6.9	**8.8**	6.5	6.8	**8.5**	6.9	7.0	**8.9**
	Mean Lap Time (s)	63.2	62.7	**48.7**	53.2	52.5	**41.1**	50.6	47.7	**38.5**	64.7	63.7	**50.2**
Head-to-Head	Safety Rate (%)	89.3	82.8	**98.1**	91.8	78.6	**97.5**	87.2	76.7	**97.1**	92.8	81.5	**97.9**
	Overtake Rate (%)	86.1	62.8	**96.1**	90.6	66.1	**97.1**	83.5	69.0	**96.8**	92.4	74.6	**97.5**

achieves mean speeds and lap times comparable to those of the expert on the training track and all three test tracks, demonstrating effective imitation of the expert's driving behavior and generalization across unseen environments.

The PPO policy surpasses both the map-based expert and the BC policy, achieving substantially higher mean speeds and shorter lap times on every track. These gains arise from PPO's continued adaptation during direct interaction with the environment, shaped by a progress reward that encourages faster driving and a collision penalty that discourages unsafe actions. Through this learning process, the policy discovers racing strategies that are often more competitive than those demonstrated by the expert and can closely resemble techniques used by human racers. One example is corner-exit track-out, in which the vehicle moves from the inside to the outside of a curve to reduce speed loss. Another example is corner-exit acceleration and corner-entry braking, in which the vehicle accelerates immediately upon exiting a curve, maintains high speed along the following straight, and brakes sharply just before entering the next curve [42].

C. Head-to-Head Racing

Table I (bottom) compares head-to-head racing performance across the three methods, evaluated using the overtake and safety rates. Here, the overtake rate is defined as the proportion of scenarios ending in a successful overtake, while the safety rate includes both successful overtakes and collision-free car-following outcomes. The map-based expert achieves a safety rate of approximately 90% across all tracks, with overtaking accounting for nearly all collision-free outcomes. This predominance of overtaking is expected because the opponent is deliberately assigned lower speeds to elicit ego-vehicle overtaking demonstrations. The remaining collisions mainly result from the expert's failure to account for opponent motion throughout an overtaking maneuver, as discussed in Sec. III-C.

Compared to the map-based expert, the BC policy adopts a substantially more conservative racing strategy, achieving lower overtake rates across all four tracks. However, this conservatism does not translate into improved safety, as its safety rate also declines on every circuit (Table I). We attribute this outcome to the imitation learning objective itself: while the network accurately mimics demonstrated control commands, supervised action matching does not establish the internal representations and tactical reasoning essential for overtaking. In closed-loop execution, minor prediction errors compound over time, driving the policy into out-of-distribution interaction states [23]. Consequently, the policy remains effective for single-vehicle racing but struggles in dense multi-vehicle interactions.

PPO significantly outperforms both the map-based expert and BC in safety and overtake rates across all evaluated tracks, demonstrating consistent generalization across diverse track layouts. This indicates that PPO not only masters high-speed racing but also acquires complex interactive capabilities that supervised action matching fails to develop. Representative examples of these interactive behaviors are examined in the following subsection.

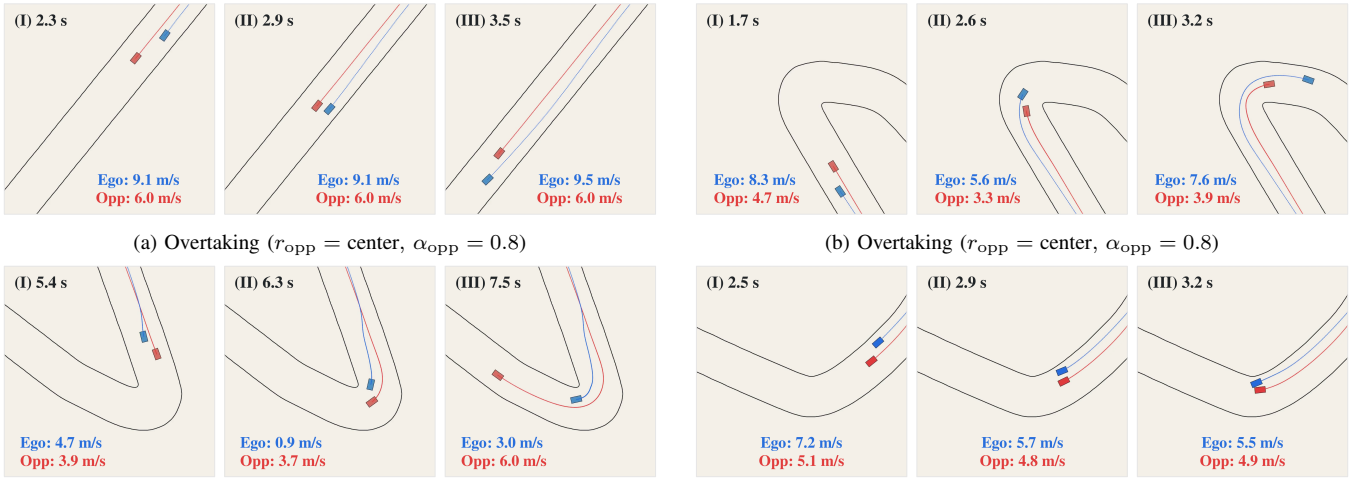
D. Scenario Illustration

Figure 5 illustrates four representative interaction scenarios recorded during head-to-head racing, covering overtaking, car-following, and collision cases. For consistency, all examples are drawn from the Austin circuit, reflecting the typical interactive behaviors observed throughout our benchmark evaluations (Table I).

Overtaking: Fig. 5(a) illustrates an overtaking sequence along a straight track segment. Upon observing the slower opponent ahead, the ego vehicle initiates a lateral shift in advance while sustaining cruising speed (Fig. 5(a-I)). As it draws alongside the opponent, the policy maintains a stable lateral margin throughout the pass (Fig. 5(a-II)). Once past the opponent, the ego vehicle smoothly returns to the track center, avoiding an abrupt cut-back (Fig. 5(a-III)).

Fig. 5(b) depicts an outside pass through a sharp curve. Approaching the turn at cruising speed, the ego vehicle follows the outside line behind the slower opponent, preparing to overtake (Fig. 5(b-I)). Entering the curve, the ego vehicle maintains the outer line with moderate braking, leveraging its speed advantage to execute the pass (Fig. 5(b-II)). Before clearing the curve, the ego vehicle accelerates early to build exit speed, pulling ahead of the opponent to complete the maneuver (Fig. 5(b-III)).

Car-Following: Fig. 5(c) depicts car-following behavior under spatial constraints. In Fig. 5(c-I), the ego vehicle prepares an inside overtake behind the opponent. However, upon entering the curve, the opponent completely blocks the inner corridor, forcing the ego vehicle to brake heavily and almost come to a complete stop (Fig. 5(c-II)). After the opponent advances through the curve, the ego vehicle accelerates to resume its driving (Fig. 5(c-III)).



(a) Overtaking ($r_{\text{opp}} = \text{center}$, $\alpha_{\text{opp}} = 0.8$)

(b) Overtaking ($r_{\text{opp}} = \text{center}$, $\alpha_{\text{opp}} = 0.8$)

(c) Car-following ($r_{\text{opp}} = \text{right}$, $\alpha_{\text{opp}} = 0.8$)

(d) Collision ($r_{\text{opp}} = \text{center}$, $\alpha_{\text{opp}} = 0.8$)

Fig. 5. Representative head-to-head interaction scenarios across time on the Austin circuit. Blue denotes the ego vehicle and red denotes the opponent, with historical trajectories rendered in matching colors. Each frame indicates the elapsed time and vehicle speeds.

TABLE II
ABLATION STUDY ON THE AUSTIN CIRCUIT

Variant	Single-Vehicle		Head-to-Head	
	Speed (m/s)	Lap Time (s)	Overtake (%)	Safety (%)
No Norm.	–	–	16.3	21.1
Linear Norm.	6.4	64.9	47.0	73.3
MLP	**6.9**	**60.9**	67.6	70.4
Transformer	6.6	63.8	**73.1**	80.6
GRU (BC)	6.7	62.7	62.8	**82.8**

Collision: Fig. 5(d) depicts a collision case. In Fig. 5(d-I), the ego vehicle prepares to overtake behind the opponent. In Fig. 5(d-II), the vehicles travel side by side as the track suddenly narrows. Left with no room along the boundary side, the ego vehicle has no choice but to steer toward the opponent, resulting in a collision (Fig. 5(d-III)).

E. Ablation Studies

We conduct an ablation study on the Austin circuit to examine the effect of different design choices in the model architecture (Table II). The evaluation covers four variants across LiDAR normalization and temporal sequence modeling. All models are trained under identical settings using behavior cloning only and achieve loss convergence.

1) LiDAR Normalization: We compare the proposed sigmoid normalization with two alternatives: no normalization and linear normalization, using the same GRU architecture. The unnormalized model fails to complete a single-vehicle lap. Linear normalization provides a viable policy, but leaves a substantial head-to-head gap, with marked drops in both overtake and safety rates. This comparison reveals that although linear normalization yields a viable policy, the sigmoid formulation uniquely provides the proximity representation required for safe and effective racing.

2) Temporal Information: To assess the importance of temporal information, we evaluate a feedforward MLP that operates only on the latest observation. The MLP replaces the recurrent unit with a $210 \rightarrow 420$ linear layer, followed by the action head. The resulting policy exhibits heuristic driving behavior, achieving a higher vehicle speed and overtake rate than the baseline. However, its safety rate drops substantially, demonstrating that temporal cues are essential for continuous interaction modeling and adaptive racing behavior.

3) Temporal Modeling: We evaluate a Transformer [43] as an alternative temporal model. The model operates on a maximum history sequence of 20 tokens, which corresponds to a span of 0.5 s. Each token represents one timestep and has a dimension of 210. With standard positional embeddings added, the tokens are processed through two encoder layers with bidirectional self-attention. The number of attention heads is set to three, each with dimension 70. The feed-forward network has a hidden dimension of 420, matching the GRU state size. The output of the latest token feeds into the action head. Although the Transformer achieves a higher overtake rate, it exhibits a slightly lower safety rate than the baseline. Due to the quadratic cost of self-attention, the model incurs *four times the inference latency* of the GRU. To balance safety, overtaking, and real-time responsiveness, we retain the GRU as our default architecture. Meanwhile, we recognize the Transformer as a practical alternative.

V. CONCLUSION

In this work, we introduce End2Race, an end-to-end learning framework for autonomous racing. Our approach achieves high computational efficiency in single-vehicle settings and adaptive decision-making during head-to-head overtaking. Collectively, these results establish End2Race as a competitive autonomous racing solution. Future work will focus on deployment to a physical vehicle and participation in F1Tenth competitions.

REFERENCES

[1] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1tenth: An open-source evaluation environment for continuous control and

reinforcement learning," *Proceedings of Machine Learning Research*, vol. 123, 2020.

[2] Hokuyo Automatic Co., Ltd., *UST-10LX Specification*, 2015.

[3] Intel Corporation, *Intel RealSense D400 Series Product Family Datasheet*, 2022.

[4] M. O'Kelly, V. Sukhil, H. Abbas, J. Harkins, C. Kao, Y. V. Pant, R. Mangharam, D. Agarwal, M. Behl, P. Burgio, and M. Bertogna, "F1/10: An open-source autonomous cyber-physical platform," *arXiv preprint arXiv:1901.08567*, 2019.

[5] S. Macenski and I. Jambrecic, "Slam toolbox: Slam for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.

[6] A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann, "Minimum curvature trajectory planning and control for an autonomous race car," *Vehicle System Dynamics*, vol. 58, no. 10, pp. 1497–1527, 2020.

[7] C. Walsh and S. Karaman, "Cddt: Fast approximate 2d ray casting for accelerated localization," *arXiv preprint arXiv:1705.01167*, 2017.

[8] M. Pivtoraiko, R. A. Knepper, and A. Kelly, "Differentially constrained mobile robot motion planning in state lattices," *Journal of Field Robotics*, vol. 26, pp. 308 – 333, March 2009.

[9] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Tech. Rep. CMU-RI-TR-92-01, Carnegie Mellon University, Pittsburgh, PA, January 1992.

[10] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, "Predictive active steering control for autonomous vehicle systems," *IEEE Transactions on Control Systems Technology*, vol. 15, no. 3, pp. 566–580, 2007.

[11] Z. Zang, H. Zheng, J. Betz, and R. Mangharam, "Local_INN: Implicit map representation and localization with invertible neural networks," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11742–11748, 2023.

[12] F. Muzzini, N. Capodieci, F. Ramanzin, and P. Burgio, "Gpu implementation of the frenet path planner for embedded autonomous systems: A case study in the F1TENTH scenario," *Journal of Systems Architecture*, vol. 154, p. 103239, 2024.

[13] V. Sezer and M. Gokasan, "A novel obstacle avoidance algorithm: "follow the gap method"," *Robotics and Autonomous Systems*, vol. 60, no. 9, pp. 1123–1134, 2012.

[14] T. Hossain, H. Habibullah, R. Islam, and R. V. Padilla, "Local path planning for autonomous mobile robots by integrating modified dynamic-window approach and improved follow the gap method," *Journal of Field Robotics*, vol. 39, no. 4, pp. 371–386, 2022.

[15] X. Sun, M. Zhou, Z. Zhuang, S. Yang, J. Betz, and R. Mangharam, "A benchmark comparison of imitation learning-based control policies for autonomous racing," in *2023 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1–5, 2023.

[16] A. Brunnbauer, L. Berducci, A. Brandstátter, M. Lechner, R. Hasani, D. Rus, and R. Grosu, "Latent imagination facilitates zero-shot transfer in autonomous racing," in *2022 International Conference on Robotics and Automation (ICRA)*, pp. 7513–7520, 2022.

[17] M. M. Zarrar, Q. Weng, B. Yerjan, A. Soyyigit, and H. Yun, "Tinylidarnet: 2d lidar-based end-to-end deep learning model for f1tenth autonomous racing," 2024.

[18] J. Zhang and H. Loidl, "F1tenth: An over-taking algorithm using machine learning," in *2023 28th International Conference on Automation and Computing (ICAC)*, pp. 01–06, 2023.

[19] E. Steiner, D. van der Spuy, F. Zhou, A. Pama, M. Liarokapis, and H. Williams, "Accelerating real-world overtaking in f1tenth racing employing reinforcement learning methods," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4528–4534, IEEE, 2025.

[20] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.

[21] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[22] C. Sammut, "Behavioral cloning," 01 2011.

[23] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pp. 627–635, JMLR Workshop and Conference Proceedings, 2011.

[24] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "Hg-dagger: Interactive imitation learning with human experts," in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 8077–8083, IEEE, 2019.

[25] J. Spencer, S. Choudhury, M. Barnes, M. Schmittle, M. Chiang, P. Ramadge, and S. Srinivasa, "Expert intervention learning: An online framework for robot learning from explicit and implicit human feedback," *Autonomous Robots*, vol. 46, no. 1, pp. 99–113, 2022.

[26] M. Paluch, F. Bolli, X. Deng, A. Rios Navarro, C. Gao, and T. Delbruck, "Fpga hardware neural control of cartpole and f1tenth race car," 2024.

[27] N. Hamilton, P. Musau, D. M. Lopez, and T. T. Johnson, "Zero-shot policy transfer in autonomous racing: Reinforcement learning vs imitation learning," in *2022 IEEE International Conference on Assured Autonomy (ICAA)*, pp. 11–20, 2022.

[28] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *International Conference on Learning Representations*, 2016.

[29] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*, pp. 1861–1870, Pmlr, 2018.

[30] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*, pp. 1587–1596, PMLR, 2018.

[31] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," *arXiv preprint arXiv:1912.01603*, 2019.

[32] T. Dwivedi, T. Betz, F. Sauerbeck, P. Manivannan, and M. Lienkamp, "Continuous control of autonomous vehicles using plan-assisted deep reinforcement learning," in *2022 22nd International Conference on Control, Automation and Systems (ICCAS)*, pp. 244–250, 2022.

[33] J. Bhargav, J. Betz, H. Zheng, and R. Mangharam, "Track based offline policy learning for overtaking maneuvers with autonomous racecars," 2021.

[34] R. Trumpp, E. Javanmardi, J. Nakazato, M. Tsukada, and M. Caccamo, "Racemop: Mapless online path planning for multi-agent autonomous racing using residual policy learning," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 8449–8456, 2024.

[35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[36] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, "Optimal trajectory generation for dynamic street scenarios in a frenét frame," in *2010 IEEE International Conference on Robotics and Automation*, pp. 987–993, 2010.

[37] H. Zheng, Z. Zhuang, J. Betz, and R. Mangharam, "Game-theoretic objective space planning," 2023.

[38] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, "PythonRobotics: A python code collection of robotics algorithms," 2018.

[39] N. Baumann, E. Ghignone, C. Hu, B. Hildisch, T. Hämmerle, A. Bettoni, A. Carron, L. Xie, and M. Magno, "Predictive spliner: Data-driven overtaking in autonomous racing using opponent trajectory prediction," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1816–1823, 2025.

[40] R. J. Bufacchi and G. D. Iannetti, "An action field theory of peripersonal space," *Trends in Cognitive Sciences*, vol. 22, no. 12, pp. 1076–1090, 2018.

[41] R. Geirhos, J. Jörn-Henrik, C. Michaelis, R. Zemel, B. Wieland, B. Matthias, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, pp. 665–673, 11 2020.

[42] W. C. Mitchell, R. Schroer, and D. B. Grisez, "Driving the traction circle," Tech. Rep. 2004-01-3545, SAE International, 2004.

[43] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.